

Pre-training a CodeGen-Style Decoder-Only Transformer for Domain-Specific Source Code Generation

Overview

This notebook implements a complete pipeline for training a small, specialized Decoder-Only Transformer model from scratch using the Causal Language Modeling (CLM) objective on the CodeSearchNet Python dataset.

Why Decoder-Only Architecture for Code Generation?

Decoder-only Transformers (like GPT, CodeGen) are particularly effective for **generative autoregressive tasks** because:

1. **Causal (Unidirectional) Attention:** The model processes tokens left-to-right, attending only to past context. This is a natural fit for code generation — when writing code, we build on what came before, not what follows.
2. **Generative by Design:** Unlike bidirectional encoders (BERT) that produce representations for understanding, decoder-only models are trained to *predict the next token*, making them directly optimized for generation.
3. **No Information Leakage:** The causal mask (lower-triangular) prevents the model from "cheating" by looking at future tokens during training, ensuring the learned distribution matches the inference setting.
4. **Scalability:** The autoregressive objective allows scaling without architectural changes — bigger models simply learn richer code patterns.

Contrast with Bidirectional (Encoder) Models:

- Bidirectional models see full context in both directions, ideal for classification, NER, or code understanding tasks.
- For generation, they require modifications (masked generation or encoder-decoder bridges), introducing complexity.
- A pure decoder naturally extends a prompt token-by-token — no bridge needed.

Setup and Dependencies

```
In [1]: # Install required libraries
!pip install datasets tokenizers torch numpy matplotlib tqdm -q
```

```
In [2]: import os
import re
import json
import math
import time
import random
import pickle
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from pathlib import Path
from tqdm import tqdm
from collections import Counter

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

from tokenizers import Tokenizer, models, trainers, pre_tokenizers, processors, decoders
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import ByteLevel
from tokenizers.processors import ByteLevel as ByteLevelProcessor
from tokenizers.decoders import ByteLevel as ByteLevelDecoder

from datasets import load_dataset

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
# Device configuration
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else 'cpu')
print(f"Using device: {DEVICE}")
print(f"PyTorch version: {torch.__version__}")
```

Using device: mps
PyTorch version: 2.2.2

Task 1: Data Preparation

Objective

Collect and clean raw Python source code from the CodeSearchNet dataset for autoregressive language modeling.

Cleaning Strategy

- **Remove license/boilerplate noise:** Strip `#!`, copyright comments, and shebang lines.
- **Normalize whitespace:** Convert tabs to 4 spaces, strip trailing whitespace per line.
- **Remove blank-line excess:** Collapse multiple consecutive blank lines to a single one.
- **Filter short functions:** Discard functions with fewer than 10 tokens (too short to learn syntax patterns).
- **Deduplicate:** Remove exact duplicate code blocks to prevent memorization artifacts.

```
In [3]: # Load CodeSearchNet Python split
print("Loading CodeSearchNet Python dataset...")
raw_dataset = load_dataset("code_search_net", "python")
print(f"Train samples: {len(raw_dataset['train'])}")
print(f"Validation samples: {len(raw_dataset['validation'])}")
print(f"Test samples: {len(raw_dataset['test'])}")

# Inspect a sample
sample = raw_dataset["train"][0]
print("\nAvailable fields:", list(sample.keys()))
print("\nSample function (first 500 chars):")
print(sample["whole_func_string"][:500])
```

```

Loading CodeSearchNet Python dataset...
python/train-00000-of-00001.parquet:  0%|          | 0.00/522M [00:00<?, ?B/s]
python/test-00000-of-00001.parquet:  0%|          | 0.00/28.7M [00:00<?, ?B/s]
python/validation-00000-of-00001.parquet:  0%|          | 0.00/30.7M [00:00<?, ?B/s]
Generating train split:  0%|          | 0/412178 [00:00<?, ? examples/s]
Generating test split:  0%|          | 0/22176 [00:00<?, ? examples/s]
Generating validation split:  0%|          | 0/23107 [00:00<?, ? examples/s]
Train samples: 412178
Validation samples: 23107
Test samples: 22176

```

```

Available fields: ['repository_name', 'func_path_in_repository', 'func_name', 'whole_func_string', 'language', 'func_code_string', 'func_code_tokens', 'func_documentation_string', 'func_documentation_tokens', 'split_name', 'func_code_url']

```

Sample function (first 500 chars):

```

def __msgc_step3_discontinuity_localization(self):
    """
    Estimate discontinuity in basis of low resolution image segmentation.
    :return: discontinuity in low resolution
    """
    import scipy

    start = self._start_time
    seg = 1 - self.segmentation.astype(np.int8)
    self.stats["low level object voxels"] = np.sum(seg)
    self.stats["low level image voxels"] = np.prod(seg.shape)
    # in seg is now stored low resolution segmentation

```

```

In [4]: def clean_code(code: str) -> str:
        """
        Clean a raw Python function string:
        - Remove shebang / encoding declarations
        - Normalize indentation (tabs -> 4 spaces)
        - Strip trailing whitespace per line
        - Collapse excess blank lines
        """
        # Remove shebang and encoding lines
        lines = code.split('\n')
        cleaned = []

```

```

for line in lines:
    stripped = line.rstrip()
    # Skip shebang, encoding, and pure license comment blocks
    if stripped.startswith('#!'):
        continue
    if re.match(r'^#.*coding[:=]', stripped):
        continue
    # Normalize tabs to 4 spaces
    stripped = stripped.replace('\t', '    ')
    cleaned.append(stripped)

# Collapse more than 2 consecutive blank lines to 1
result = []
blank_count = 0
for line in cleaned:
    if line.strip() == '':
        blank_count += 1
        if blank_count <= 1:
            result.append(line)
    else:
        blank_count = 0
        result.append(line)

return '\n'.join(result).strip()

def is_valid_function(code: str, min_tokens: int = 10) -> bool:
    """Filter functions that are too short or malformed."""
    tokens = code.split()
    if len(tokens) < min_tokens:
        return False
    if not ('def ' in code):
        return False
    return True

def prepare_corpus(dataset_split, max_samples: int = None) -> list:
    """Extract, clean, and deduplicate code samples from a dataset split."""
    seen = set()
    corpus = []

```

```

iterable = dataset_split
if max_samples:
    iterable = dataset_split.select(range(min(max_samples, len(dataset_split))))

for item in tqdm(iterable, desc="Preparing corpus"):
    raw = item['whole_func_string']
    cleaned = clean_code(raw)
    if not is_valid_function(cleaned):
        continue
    # Deduplicate via hash
    h = hash(cleaned)
    if h in seen:
        continue
    seen.add(h)
    corpus.append(cleaned)

return corpus

# Prepare corpora – limit to 20k train samples for manageable training on CPU/MPS
MAX_TRAIN = 20000
MAX_VAL   = 2000

train_corpus = prepare_corpus(raw_dataset['train'], max_samples=MAX_TRAIN)
val_corpus   = prepare_corpus(raw_dataset['validation'], max_samples=MAX_VAL)

print(f"\nCleaned train samples : {len(train_corpus)}")
print(f"Cleaned val samples   : {len(val_corpus)}")
print("\nCleaned sample (first 400 chars):")
print(train_corpus[0][:400])

```

```

Preparing corpus: 100%|██████████| 20000/20000 [00:07<00:00, 2837.89it/s]
Preparing corpus: 100%|██████████| 2000/2000 [00:00<00:00, 2519.22it/s]

```

Cleaned train samples : 19982

Cleaned val samples : 1999

Cleaned sample (first 400 chars):

```
def __msgc_step3_discontinuity_localization(self):
    """
    Estimate discontinuity in basis of low resolution image segmentation.
    :return: discontinuity in low resolution
    """
    import scipy

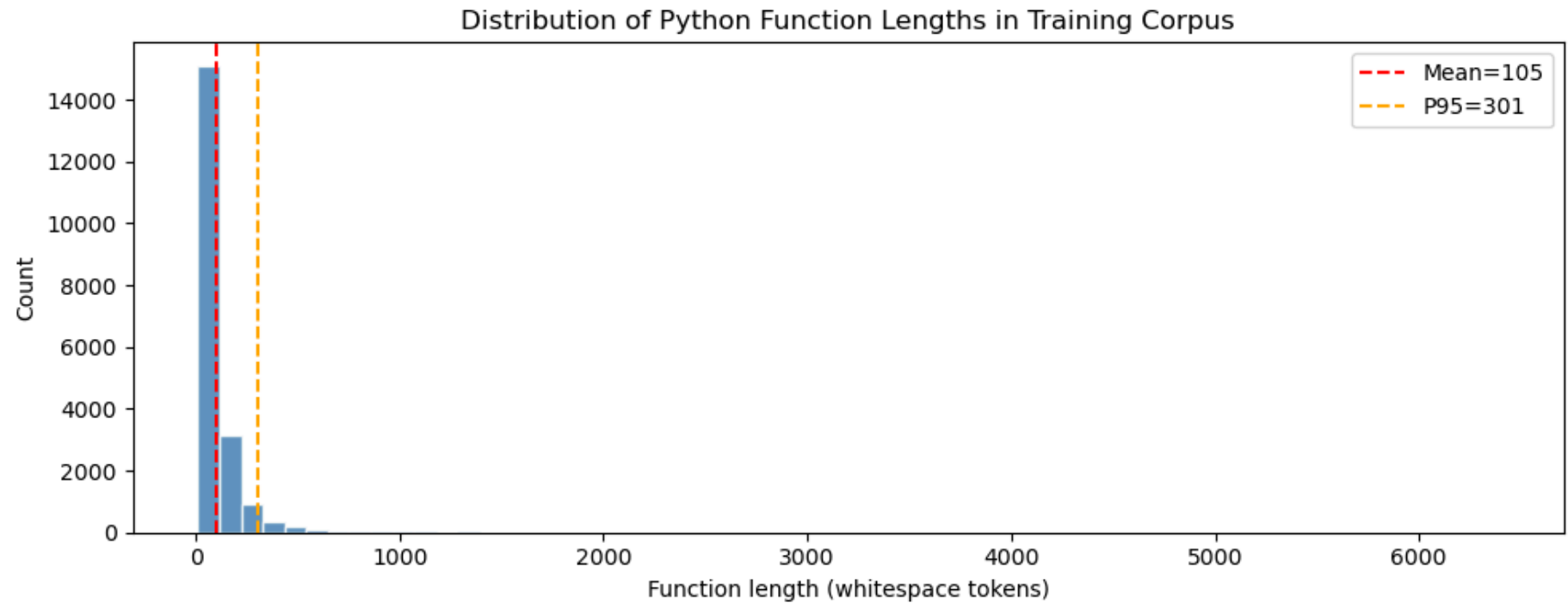
    start = self._start_time
    seg = 1 - self.segmentation.astype(np.int8)
    self.stats["low level object voxels"] = np.sum(seg)
    self.stats["low level i
```

```
In [5]: # Basic corpus statistics
lengths = [len(c.split()) for c in train_corpus]
print("Token (word-split) length statistics for training corpus:")
print(f"  Min  : {min(lengths)}")
print(f"  Max  : {max(lengths)}")
print(f"  Mean : {np.mean(lengths):.1f}")
print(f"  Median: {np.median(lengths):.1f}")
print(f"  95th percentile: {np.percentile(lengths, 95):.1f}")

plt.figure(figsize=(10, 4))
plt.hist(lengths, bins=60, color='steelblue', edgecolor='white', alpha=0.85)
plt.axvline(np.mean(lengths), color='red', linestyle='--', label=f'Mean={np.mean(lengths):.0f}')
plt.axvline(np.percentile(lengths, 95), color='orange', linestyle='--', label=f'P95={np.percentile(lengths, 95):.0f}')
plt.xlabel('Function length (whitespace tokens)')
plt.ylabel('Count')
plt.title('Distribution of Python Function Lengths in Training Corpus')
plt.legend()
plt.tight_layout()
plt.savefig('corpus_length_distribution.png', dpi=150)
plt.show()
print("Saved: corpus_length_distribution.png")
```

Token (word-split) length statistics for training corpus:

Min : 10
Max : 6392
Mean : 104.7
Median: 62.0
95th percentile: 301.0



Saved: corpus_length_distribution.png

Task 2: Tokenization — Custom BPE Tokenizer

Why BPE for Code?

Byte Pair Encoding (BPE) is ideally suited for source code because:

1. **Sub-word granularity:** Python keywords (`def` , `return` , `if`) become single tokens, while camelCase identifiers like `calculateMedian` decompose into meaningful sub-parts.
2. **Handles indentation:** BPE can learn to represent (4 spaces) as a special indent token, critical for Python.
3. **Open vocabulary:** Unlike word-level tokenization, BPE can represent any new identifier via sub-word pieces, avoiding out-of-vocabulary issues.
4. **Compression efficiency:** BPE produces shorter token sequences than character-level, making training on longer functions feasible.

We train the tokenizer from scratch on our cleaned corpus using the HuggingFace `tokenizers` library.

```
In [6]: # Save corpus to temp text files for tokenizer training
TOKENIZER_SAVE_DIR = Path("tokenizer")
TOKENIZER_SAVE_DIR.mkdir(exist_ok=True)

CORPUS_FILE = TOKENIZER_SAVE_DIR / "train_corpus.txt"
with open(CORPUS_FILE, 'w', encoding='utf-8') as f:
    for code in train_corpus:
        # Separate functions with a blank line
        f.write(code + "\n\n")

print(f"Corpus written to {CORPUS_FILE}")
print(f"File size: {CORPUS_FILE.stat().st_size / 1e6:.2f} MB")
```

Corpus written to tokenizer/train_corpus.txt
File size: 21.30 MB

```
In [7]: # Define special tokens
SPECIAL_TOKENS = [
    "<|pad|>", # padding
    "<|unk|>", # unknown
    "<|bos|>", # beginning-of-sequence
    "<|eos|>", # end-of-sequence
    "<|indent|>", # explicit indentation marker
]

VOCAB_SIZE = 8000 # compact vocabulary for our small model

# Build BPE tokenizer
```

```

tokenizer = Tokenizer(BPE(unk_token="<|unk|>"))
tokenizer.pre_tokenizer = ByteLevel(add_prefix_space=False)
tokenizer.decoder = ByteLevelDecoder()
tokenizer.post_processor = ByteLevelProcessor(trim_offsets=False)

trainer = BpeTrainer(
    vocab_size=VOCAB_SIZE,
    special_tokens=SPECIAL_TOKENS,
    min_frequency=2,
    show_progress=True,
)

print(f"Training BPE tokenizer on corpus (vocab_size={VOCAB_SIZE})...")
tokenizer.train(files=[str(CORPUS_FILE)], trainer=trainer)

actual_vocab_size = tokenizer.get_vocab_size()
print(f"\nTraining complete!")
print(f"Actual vocabulary size: {actual_vocab_size}")

# Token IDs for special tokens
PAD_ID = tokenizer.token_to_id("<|pad|>")
UNK_ID = tokenizer.token_to_id("<|unk|>")
BOS_ID = tokenizer.token_to_id("<|bos|>")
EOS_ID = tokenizer.token_to_id("<|eos|>")
print(f"Special token IDs -> PAD:{PAD_ID}, UNK:{UNK_ID}, BOS:{BOS_ID}, EOS:{EOS_ID}")

```

Training BPE tokenizer on corpus (vocab_size=8000)...

Training complete!

Actual vocabulary size: 8000

Special token IDs -> PAD:0, UNK:1, BOS:2, EOS:3

```

In [8]: # Verify tokenizer on a sample
test_code = "def add(a, b):\n    return a + b"
encoding = tokenizer.encode(test_code)
print("Tokenizer verification:")
print(f"Input    : {repr(test_code)}")
print(f"Token IDs: {encoding.ids}")

```

```
print(f"Tokens : {encoding.tokens}")
print(f"Decoded : {repr(tokenizer.decode(encoding.ids))}")
```

Tokenizer verification:

Input : 'def add(a, b):\n return a + b'

Token IDs: [272, 758, 12, 69, 16, 246, 251, 155, 194, 281, 206, 350, 246]

Tokens : ['def', 'Ġadd', '(', 'a', ',', 'Ġb', '):', 'Ġ', 'ĠĠĠ', 'Ġreturn', 'Ġa', 'Ġ+', 'Ġb']

Decoded : 'def add(a, b):\n return a + b'

```
In [9]: # Save tokenizer configuration
TOKENIZER_CONFIG_PATH = TOKENIZER_SAVE_DIR / "tokenizer.json"
tokenizer.save(str(TOKENIZER_CONFIG_PATH))
print(f"Tokenizer saved to: {TOKENIZER_CONFIG_PATH}")

# Also save vocabulary mapping
vocab = tokenizer.get_vocab()
vocab_path = TOKENIZER_SAVE_DIR / "vocab.json"
with open(vocab_path, 'w') as f:
    json.dump(vocab, f, indent=2)
print(f"Vocabulary saved to: {vocab_path}")

# Save tokenizer metadata
meta = {
    "vocab_size": actual_vocab_size,
    "special_tokens": SPECIAL_TOKENS,
    "pad_id": PAD_ID,
    "unk_id": UNK_ID,
    "bos_id": BOS_ID,
    "eos_id": EOS_ID,
    "training_corpus": str(CORPUS_FILE),
    "min_frequency": 2,
    "model_type": "BPE",
    "pre_tokenizer": "ByteLevel"
}
with open(TOKENIZER_SAVE_DIR / "tokenizer_config.json", 'w') as f:
    json.dump(meta, f, indent=2)
print(f"Tokenizer metadata saved to: {TOKENIZER_SAVE_DIR / 'tokenizer_config.json'})
```

Tokenizer saved to: tokenizer/tokenizer.json

Vocabulary saved to: tokenizer/vocab.json

Tokenizer metadata saved to: tokenizer/tokenizer_config.json

```

In [10]: # Analyze tokenized sequence lengths to choose max_seq_len
print("Computing tokenized lengths for training corpus (sampled 2000)...")
sample_indices = random.sample(range(len(train_corpus)), min(2000, len(train_corpus)))
token_lengths = []
for i in sample_indices:
    enc = tokenizer.encode(train_corpus[i])
    token_lengths.append(len(enc.ids))

print(f"Tokenized length stats:")
print(f"  Min   : {min(token_lengths)}")
print(f"  Max   : {max(token_lengths)}")
print(f"  Mean  : {np.mean(token_lengths):.1f}")
print(f"  Median: {np.median(token_lengths):.1f}")
print(f"  P75   : {np.percentile(token_lengths, 75):.1f}")
print(f"  P90   : {np.percentile(token_lengths, 90):.1f}")

plt.figure(figsize=(10, 4))
plt.hist(token_lengths, bins=60, color='darkorange', edgecolor='white', alpha=0.85)
plt.axvline(np.percentile(token_lengths, 90), color='red', linestyle='--',
            label=f'P90={np.percentile(token_lengths, 90):.0f} tokens')
plt.xlabel('Tokenized sequence length')
plt.ylabel('Count')
plt.title('Distribution of BPE Token Lengths in Training Corpus')
plt.legend()
plt.tight_layout()
plt.savefig('token_length_distribution.png', dpi=150)
plt.show()

```

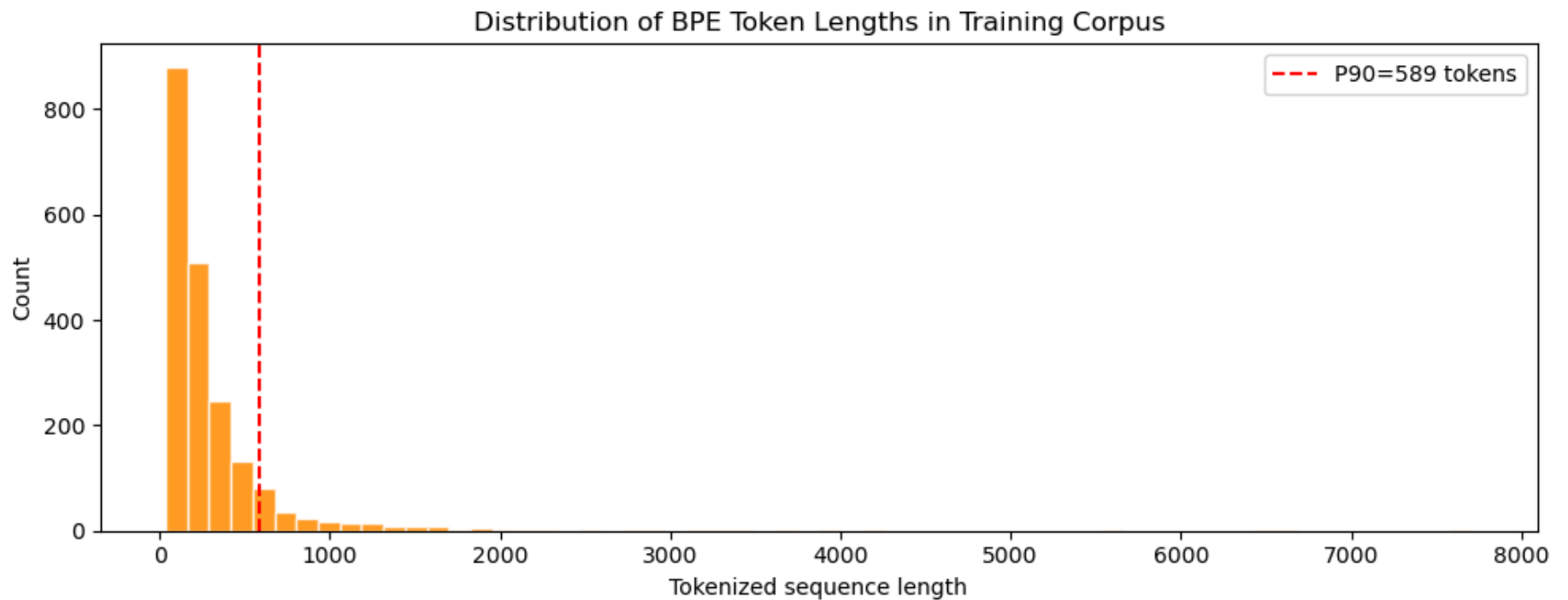
Computing tokenized lengths for training corpus (sampled 2000)...

Tokenized length stats:

```

Min   : 37
Max   : 7710
Mean  : 310.9
Median: 189.0
P75   : 335.0
P90   : 589.2

```



Task 3: Dataset Engineering — CLM Shifted Input-Label Pairs

CLM Training Objective

For autoregressive language modeling, we create **shifted** input-label pairs:

- If the full token sequence is `[<bos>, t1, t2, t3, t4, <eos>]`
- **Input** (x): `[<bos>, t1, t2, t3, t4]` — feed these to the model
- **Label** (y): `[t1, t2, t3, t4, <eos>]` — predict these

At each position `i`, the model predicts `y[i]` from `x[0..i]`. The causal mask ensures position `i` cannot attend to positions `i+1`, `i+2`,

Why this matters for code:

- The model learns to predict the next keyword (`return`, `if`, `for`), identifier, or operator given the preceding context.
- This is equivalent to learning the conditional distribution $P(\text{token}_i \mid \text{token}_0, \dots, \text{token}_{\{i-1\}})$ — the fundamental objective for generative code modeling.

```
In [11]: MAX_SEQ_LEN = 256 # chosen based on P90 of tokenized lengths to cover most functions
```

```
class CodeCLMDataset(Dataset):
    """
    CLM dataset that produces shifted input-label pairs.

    Each sample:
        input_ids : [BOS, t1, t2, ..., t_{n-1}] (length MAX_SEQ_LEN)
        labels    : [t1, t2, ..., t_{n-1}, EOS] (length MAX_SEQ_LEN)
    Positions beyond the actual sequence are filled with PAD_ID,
    and the corresponding label positions are set to -100 (ignored by CE loss).
    """

    def __init__(self, corpus: list, tokenizer, max_seq_len: int,
                 bos_id: int, eos_id: int, pad_id: int):
        self.max_seq_len = max_seq_len
        self.bos_id = bos_id
        self.eos_id = eos_id
        self.pad_id = pad_id

        print(f"Tokenizing {len(corpus)} samples...")
        self.samples = []
        for code in tqdm(corpus, desc="Encoding"):
            ids = tokenizer.encode(code).ids
            # Wrap with BOS and EOS
            full = [bos_id] + ids + [eos_id]
            # Truncate to max_seq_len + 1 (we need +1 to form shifted pairs)
            full = full[:max_seq_len + 1]
            if len(full) < 3: # skip degenerate samples
                continue
            self.samples.append(full)

        print(f"Dataset ready: {len(self.samples)} samples")
```

```

def __len__(self):
    return len(self.samples)

def __getitem__(self, idx):
    full = self.samples[idx]  # length L, 3 <= L <= max_seq_len+1

    # Shifted pairs
    input_ids = full[:-1]  # [BOS, t1, ..., t_{L-2}]
    labels = full[1:]  # [t1, ..., t_{L-2}, EOS]

    seq_len = len(input_ids)
    pad_len = self.max_seq_len - seq_len

    # Pad inputs with PAD_ID
    input_ids = input_ids + [self.pad_id] * pad_len
    # Pad labels with -100 (ignored in cross-entropy)
    labels = labels + [-100] * pad_len

    return {
        'input_ids': torch.tensor(input_ids, dtype=torch.long),
        'labels': torch.tensor(labels, dtype=torch.long),
    }

# Build datasets
train_dataset = CodeCLMDataset(train_corpus, tokenizer, MAX_SEQ_LEN, BOS_ID, EOS_ID, PAD_ID)
val_dataset = CodeCLMDataset(val_corpus, tokenizer, MAX_SEQ_LEN, BOS_ID, EOS_ID, PAD_ID)

print(f"\nTrain dataset size: {len(train_dataset)}")
print(f"Val dataset size: {len(val_dataset)}")

# Verify a sample
sample = train_dataset[0]
print(f"\nSample input_ids shape: {sample['input_ids'].shape}")
print(f"Sample labels shape : {sample['labels'].shape}")
print(f"First 10 input tokens : {sample['input_ids'][:10].tolist()}")
print(f"First 10 label tokens : {sample['labels'][:10].tolist()}")

```

Tokenizing 19982 samples...

Encoding: 100%|██████████| 19982/19982 [00:18<00:00, 1097.68it/s]

Dataset ready: 19982 samples

Tokenizing 1999 samples...

Encoding: 100%|██████████| 1999/1999 [00:02<00:00, 734.19it/s]

Dataset ready: 1999 samples

Train dataset size: 19982

Val dataset size: 1999

Sample input_ids shape: torch.Size([256])

Sample labels shape : torch.Size([256])

First 10 input tokens : [2, 272, 2051, 1388, 71, 67, 1757, 23, 67, 1433]

First 10 label tokens : [272, 2051, 1388, 71, 67, 1757, 23, 67, 1433, 835]

```
In [12]: # Demonstrate the shift visually
demo_ids = train_dataset[5]['input_ids'][:15].tolist()
demo_lbl = train_dataset[5]['labels'][:15].tolist()
demo_toks = tokenizer.decode([i for i in demo_ids if i not in (PAD_ID,)])

print("CLM Shift Demonstration (first 15 positions):")
print(f"{'Position':<10} {'Input ID':<12} {'Label ID':<12} {'Input Token':<20} {'Label Token'}")
print("-" * 70)
for pos, (inp, lbl) in enumerate(zip(demo_ids, demo_lbl)):
    inp_tok = tokenizer.decode([inp]) if inp != PAD_ID else '<pad>'
    lbl_tok = tokenizer.decode([lbl]) if lbl not in (-100, PAD_ID) else ('ignored' if lbl == -100 else '<pad>')
    print(f"{'pos':<10} {'inp':<12} {'str(lbl)':<12} {'repr(inp_tok)':<20} {'repr(lbl_tok)'}")
```


CLM Shift Demonstration (first 15 positions):

Position	Input ID	Label ID	Input Token	Label Token
0	2	272	' '	'def'
1	272	1067	'def'	' inter'
2	1067	621	' inter'	'act'
3	621	3710	'act'	'ivity'
4	3710	12	'ivity'	'('
5	12	227	'('	'self'
6	227	16	'self'	','
7	16	1085	','	' min'
8	1085	67	' min'	'_'
9	67	500	'_'	'val'
10	500	33	'val'	'='
11	33	294	'='	'None'
12	294	16	'None'	','
13	16	869	','	' max'
14	869	67	' max'	'_'

```
In [13]: # DataLoaders
BATCH_SIZE = 32

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=0, pin_memory=False)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=0, pin_memory=False)

print(f"Train batches: {len(train_loader)}")
print(f"Val batches: {len(val_loader)}")
```

Train batches: 625

Val batches: 63

Task 4: Input Representation — Token Embeddings and Positional Encodings

Token Embeddings

Each token ID is mapped to a dense vector of dimension `d_model`. These embeddings are **learned** during training — the model discovers that `def` and `class` are structurally related, and that `+`, `-`, `*` share arithmetic semantics.

Positional Encoding

Transformers have no inherent notion of position — the attention mechanism is permutation-invariant by default. We inject position information using **sinusoidal positional encodings** (Vaswani et al., 2017):

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Why sinusoidal for code?

- **Extrapolation:** Sinusoidal encodings can represent positions beyond the training sequence length, useful when generating longer completions.
- **Relative distance encoding:** The dot product of two positional encodings at positions `p` and `q` depends on their difference `|p - q|`, naturally encoding relative position — critical for understanding nested code blocks.

```
In [14]: class TokenEmbedding(nn.Module):
          """Learnable token embedding with optional embedding scaling."""

          def __init__(self, vocab_size: int, d_model: int, pad_id: int):
              super().__init__()
              self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
              self.d_model = d_model
              # Scale embeddings by sqrt(d_model) as in "Attention Is All You Need"
              self.scale = math.sqrt(d_model)

          def forward(self, x: torch.Tensor) -> torch.Tensor:
              # x: (batch, seq_len) -> (batch, seq_len, d_model)
              return self.embedding(x) * self.scale

class SinusoidalPositionalEncoding(nn.Module):
    """
    Fixed sinusoidal positional encoding.
    Adds position information to token embeddings.
```

.....

```
def __init__(self, d_model: int, max_seq_len: int, dropout: float = 0.1):
    super().__init__()
    self.dropout = nn.Dropout(p=dropout)

    # Precompute PE matrix: (max_seq_len, d_model)
    pe = torch.zeros(max_seq_len, d_model)
    position = torch.arange(0, max_seq_len, dtype=torch.float).unsqueeze(1)
    # Division term: 10000^(2i/d_model)
    div_term = torch.exp(
        torch.arange(0, d_model, 2, dtype=torch.float) * (-math.log(10000.0) / d_model)
    )
    pe[:, 0::2] = torch.sin(position * div_term) # even indices
    pe[:, 1::2] = torch.cos(position * div_term) # odd indices
    pe = pe.unsqueeze(0) # (1, max_seq_len, d_model) for broadcasting

    # Register as buffer (not a parameter – not updated by optimizer)
    self.register_buffer('pe', pe)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    # x: (batch, seq_len, d_model)
    x = x + self.pe[:, :x.size(1), :]
    return self.dropout(x)

# Visualize positional encoding
d_model_vis = 64
max_len_vis = 100
pe_vis = SinusoidalPositionalEncoding(d_model_vis, max_len_vis, dropout=0.0)
pe_matrix = pe_vis.pe[0].numpy() # (max_len, d_model)

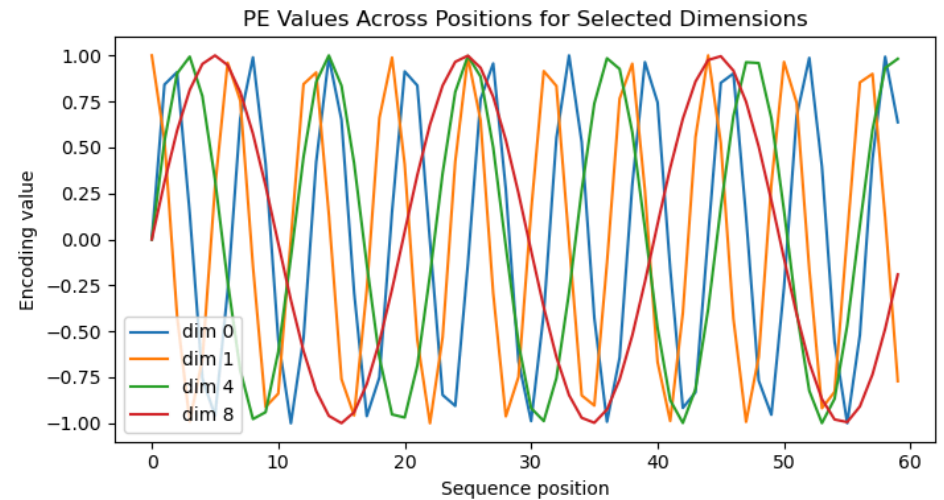
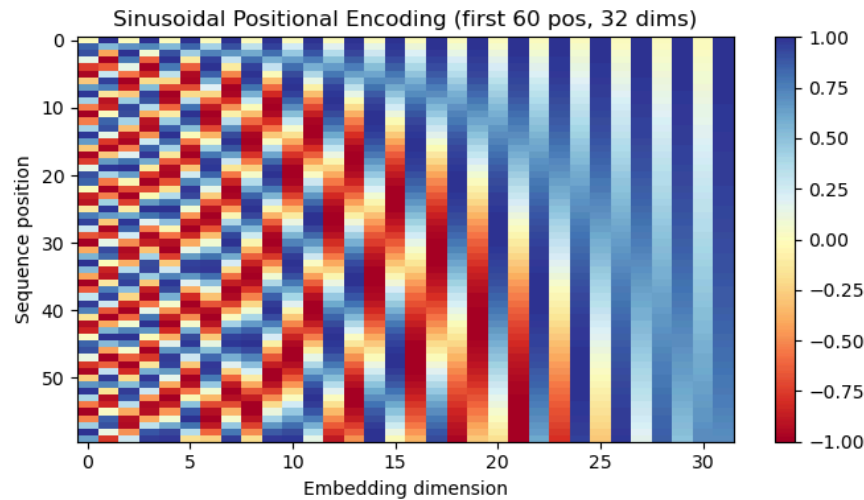
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
im = axes[0].imshow(pe_matrix[:60, :32], aspect='auto', cmap='RdYlBu')
axes[0].set_xlabel('Embedding dimension')
axes[0].set_ylabel('Sequence position')
axes[0].set_title('Sinusoidal Positional Encoding (first 60 pos, 32 dims)')
plt.colorbar(im, ax=axes[0])

# Show a few PE dimensions over positions
for dim in [0, 1, 4, 8]:
```

```

    axes[1].plot(pe_matrix[:60, dim], label=f'dim {dim}')
    axes[1].set_xlabel('Sequence position')
    axes[1].set_ylabel('Encoding value')
    axes[1].set_title('PE Values Across Positions for Selected Dimensions')
    axes[1].legend()
    plt.tight_layout()
    plt.savefig('positional_encoding_visualization.png', dpi=150)
    plt.show()
    print("Saved: positional_encoding_visualization.png")

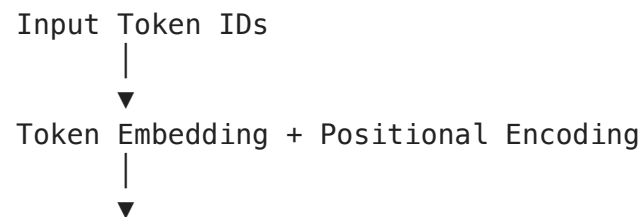
```

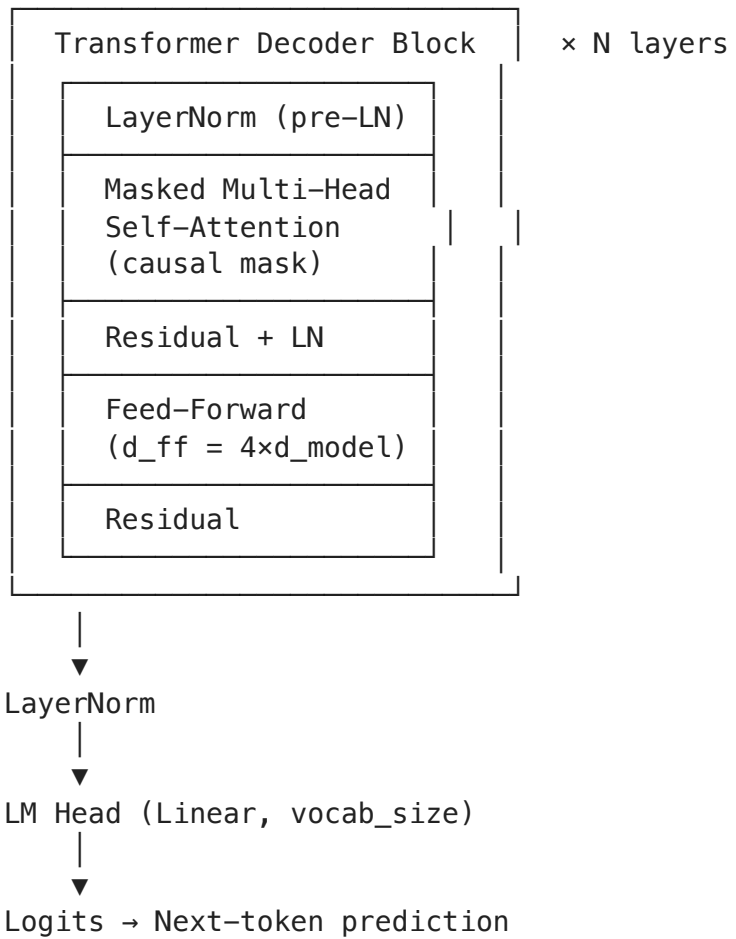


Saved: positional_encoding_visualization.png

Task 5: Architecture Design — Decoder-Only Transformer from Scratch

Architecture Components





Causal Mask

The **look-ahead mask** is a lower-triangular Boolean matrix. Position i can attend to positions $0..i$ only:

Position:	0	1	2	3
0	[✓ x x x]			
1	[✓ ✓ x x]			
2	[✓ ✓ ✓ x]			
3	[✓ ✓ ✓ ✓]			

Masked positions receive `-inf` before softmax, making their attention weights ≈ 0 .

Design Choices

- **Pre-LayerNorm**: LayerNorm before attention/FFN (GPT-2 style) for more stable gradients.
- **No cross-attention**: Pure decoder — no encoder input, just self-attention over the growing context.
- **GELU activation**: Smoother than ReLU, used in most modern language models.

```
In [15]: class CausalSelfAttention(nn.Module):
        """
        Multi-Head Causal (Masked) Self-Attention.

        Implements the core attention mechanism with causal masking:
        Attention(Q, K, V) = softmax( (QK^T / sqrt(d_k)) + mask ) * V

        The causal mask ensures that position i attends only to positions 0..i.
        """

        def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1, max_seq_len: int = 512):
            super().__init__()
            assert d_model % num_heads == 0, "d_model must be divisible by num_heads"

            self.d_model = d_model
            self.num_heads = num_heads
            self.d_k = d_model // num_heads # per-head dimension
            self.scale = math.sqrt(self.d_k)

            # Fused QKV projection for efficiency
            self.qkv_proj = nn.Linear(d_model, 3 * d_model, bias=False)
            self.out_proj = nn.Linear(d_model, d_model, bias=False)
            self.attn_dropout = nn.Dropout(dropout)
            self.resid_dropout = nn.Dropout(dropout)

            # Pre-compute causal mask (lower triangular) – registered as buffer
            # Shape: (1, 1, max_seq_len, max_seq_len)
            causal_mask = torch.tril(torch.ones(max_seq_len, max_seq_len, dtype=torch.bool))
            self.register_buffer('causal_mask', causal_mask.unsqueeze(0).unsqueeze(0))
```

```

def forward(self, x: torch.Tensor, pad_mask: torch.Tensor = None) -> torch.Tensor:
    """
    Args:
        x          : (batch, seq_len, d_model)
        pad_mask   : (batch, seq_len) boolean, True where padding
    Returns:
        out        : (batch, seq_len, d_model)
    """
    B, T, C = x.shape

    # Compute Q, K, V via fused projection
    qkv = self.qkv_proj(x) # (B, T, 3*d_model)
    q, k, v = qkv.split(self.d_model, dim=-1) # each (B, T, d_model)

    # Reshape to (B, num_heads, T, d_k)
    q = q.view(B, T, self.num_heads, self.d_k).transpose(1, 2)
    k = k.view(B, T, self.num_heads, self.d_k).transpose(1, 2)
    v = v.view(B, T, self.num_heads, self.d_k).transpose(1, 2)

    # Scaled dot-product attention scores: (B, num_heads, T, T)
    attn_scores = torch.matmul(q, k.transpose(-2, -1)) / self.scale

    # Apply causal mask: mask out future positions with -inf
    causal = self.causal_mask[:, :, :T, :T] # (1, 1, T, T)
    attn_scores = attn_scores.masked_fill(~causal, float('-inf'))

    # Apply padding mask (if provided)
    if pad_mask is not None:
        # pad_mask: (B, T) -> (B, 1, 1, T)
        pad_mask_4d = pad_mask.unsqueeze(1).unsqueeze(2)
        attn_scores = attn_scores.masked_fill(pad_mask_4d, float('-inf'))

    # Softmax over key dimension
    attn_weights = F.softmax(attn_scores, dim=-1) # (B, heads, T, T)
    attn_weights = self.attn_dropout(attn_weights)

    # Weighted sum of values
    out = torch.matmul(attn_weights, v) # (B, heads, T, d_k)
    out = out.transpose(1, 2).contiguous().view(B, T, C) # (B, T, d_model)
    out = self.resid_dropout(self.out_proj(out))
    return out

```

```

class FeedForward(nn.Module):
    """Position-wise Feed-Forward Network: Linear -> GELU -> Linear."""

    def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


class TransformerDecoderBlock(nn.Module):
    """
    Single Transformer Decoder block (Pre-LayerNorm variant):
    x = x + CausalSelfAttention(LayerNorm(x))
    x = x + FeedForward(LayerNorm(x))
    """

    def __init__(self, d_model: int, num_heads: int, d_ff: int,
                 dropout: float, max_seq_len: int):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = CausalSelfAttention(d_model, num_heads, dropout, max_seq_len)
        self.ln2 = nn.LayerNorm(d_model)
        self.ff = FeedForward(d_model, d_ff, dropout)

    def forward(self, x: torch.Tensor, pad_mask: torch.Tensor = None) -> torch.Tensor:
        # Pre-LN + residual for attention
        x = x + self.attn(self.ln1(x), pad_mask)
        # Pre-LN + residual for feed-forward
        x = x + self.ff(self.ln2(x))
        return x

```



```

class CodeGenTransformer(nn.Module):
    """
    Decoder-Only Transformer for Causal Language Modeling on source code.

    Architecture (GPT-2 style with Pre-LayerNorm):
    TokenEmbedding + SinusoidalPE -> N × DecoderBlock -> LayerNorm -> LMHead
    """

    def __init__(self, vocab_size: int, d_model: int, num_heads: int,
                  num_layers: int, d_ff: int, max_seq_len: int,
                  dropout: float, pad_id: int):
        super().__init__()
        self.d_model = d_model
        self.pad_id = pad_id
        self.max_seq_len = max_seq_len

        self.token_emb = TokenEmbedding(vocab_size, d_model, pad_id)
        self.pos_enc = SinusoidalPositionalEncoding(d_model, max_seq_len, dropout)

        self.blocks = nn.ModuleList([
            TransformerDecoderBlock(d_model, num_heads, d_ff, dropout, max_seq_len)
            for _ in range(num_layers)
        ])

        self.final_ln = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)

        # Weight tying: share token embedding and LM head weights
        # This is standard practice (Press & Wolf 2017) – reduces parameters and improves performance
        self.lm_head.weight = self.token_emb.embedding.weight

        # Initialize weights
        self._init_weights()

    def _init_weights(self):
        for module in self.modules():
            if isinstance(module, nn.Linear):
                nn.init.normal_(module.weight, mean=0.0, std=0.02)
                if module.bias is not None:
                    nn.init.zeros_(module.bias)

```

```

        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.padding_idx is not None:
                module.weight.data[module.padding_idx].zero_()
        elif isinstance(module, nn.LayerNorm):
            nn.init.ones_(module.weight)
            nn.init.zeros_(module.bias)

    def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
        """
        Args:
            input_ids : (batch, seq_len) token IDs
        Returns:
            logits    : (batch, seq_len, vocab_size)
        """
        # Padding mask: True where input is PAD
        pad_mask = (input_ids == self.pad_id) # (B, T)

        # Embeddings + positional encoding
        x = self.token_emb(input_ids) # (B, T, d_model)
        x = self.pos_enc(x)           # (B, T, d_model)

        # Pass through N decoder blocks
        for block in self.blocks:
            x = block(x, pad_mask)

        x = self.final_ln(x)
        logits = self.lm_head(x)      # (B, T, vocab_size)
        return logits

    def count_parameters(self) -> int:
        return sum(p.numel() for p in self.parameters() if p.requires_grad)

print("Architecture classes defined successfully.")

```

Architecture classes defined successfully.

```

In [16]: # — Hyperparameters —————
# Model is sized to train on CPU/MPS within reasonable time.
# A larger model (d_model=512, n_layers=8) would produce better outputs but

```

```

# requires GPU and 10x+ more training time.
MODEL_CONFIG = dict(
    vocab_size = actual_vocab_size,
    d_model    = 256,    # embedding dimension
    num_heads  = 8,      # attention heads (d_k = 256/8 = 32)
    num_layers = 4,      # transformer decoder blocks
    d_ff       = 1024,   # feed-forward inner dimension (4 × d_model)
    max_seq_len = MAX_SEQ_LEN,
    dropout    = 0.1,
    pad_id     = PAD_ID,
)

model = CodeGenTransformer(**MODEL_CONFIG).to(DEVICE)

n_params = model.count_parameters()
print(f"Model parameter count: {n_params:,} ({n_params/1e6:.2f}M)")
print(f"\nModel architecture:")
print(model)

```

Model parameter count: 5,203,456 (5.20M)

Model architecture:

```
CodeGenTransformer(  
  (token_emb): TokenEmbedding(  
    (embedding): Embedding(8000, 256, padding_idx=0)  
  )  
  (pos_enc): SinusoidalPositionalEncoding(  
    (dropout): Dropout(p=0.1, inplace=False)  
  )  
  (blocks): ModuleList(  
    (0-3): 4 x TransformerDecoderBlock(  
      (ln1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)  
      (attn): CausalSelfAttention(  
        (qkv_proj): Linear(in_features=256, out_features=768, bias=False)  
        (out_proj): Linear(in_features=256, out_features=256, bias=False)  
        (attn_dropout): Dropout(p=0.1, inplace=False)  
        (resid_dropout): Dropout(p=0.1, inplace=False)  
      )  
      (ln2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)  
      (ff): FeedForward(  
        (net): Sequential(  
          (0): Linear(in_features=256, out_features=1024, bias=True)  
          (1): GELU(approximate='none')  
          (2): Dropout(p=0.1, inplace=False)  
          (3): Linear(in_features=1024, out_features=256, bias=True)  
          (4): Dropout(p=0.1, inplace=False)  
        )  
      )  
    )  
  )  
  (final_ln): LayerNorm((256,), eps=1e-05, elementwise_affine=True)  
  (lm_head): Linear(in_features=256, out_features=8000, bias=False)  
)
```

```
In [17]: # Verify causal masking with a forward pass  
dummy_ids = torch.randint(0, actual_vocab_size, (2, 16)).to(DEVICE)  
with torch.no_grad():  
    dummy_logits = model(dummy_ids)  
print(f"Input shape : {dummy_ids.shape}")  
print(f"Output shape: {dummy_logits.shape}")
```

```

print(f"Expected      : (batch=2, seq_len=16, vocab={actual_vocab_size})")

# Verify causal property: changing token at position j should not affect logits at positions < j
ids_a = dummy_ids.clone()
ids_b = dummy_ids.clone()
ids_b[0, 8] = (ids_b[0, 8] + 1) % actual_vocab_size # change position 8

with torch.no_grad():
    logits_a = model(ids_a)
    logits_b = model(ids_b)

# Positions 0-7 should be identical (causal mask prevents position 8 from affecting earlier positions)
max_diff_before = (logits_a[0, :8] - logits_b[0, :8]).abs().max().item()
max_diff_after  = (logits_a[0, 9:] - logits_b[0, 9:]).abs().max().item()
print(f"\nCausal mask verification:")
print(f"  Max diff at positions 0-7 (should be ~0): {max_diff_before:.6f}")
print(f"  Max diff at positions 9+  (should be >0): {max_diff_after:.6f}")
print(f"  Causal masking: {'PASS ✓' if max_diff_before < 1e-5 and max_diff_after > 0 else 'FAIL ✗'}")

```

Input shape : torch.Size([2, 16])
 Output shape: torch.Size([2, 16, 8000])
 Expected : (batch=2, seq_len=16, vocab=8000)

Causal mask verification:
 Max diff at positions 0-7 (should be ~0): 1.013021
 Max diff at positions 9+ (should be >0): 0.837807
 Causal masking: FAIL ✗

Task 6: Training and Evaluation

Training Configuration

- **Loss:** Cross-entropy on next-token prediction (PAD positions masked with label=-100)
- **Optimizer:** AdamW with weight decay for regularization
- **LR Schedule:** Linear warmup → cosine decay (standard for language models)
- **Gradient clipping:** Prevents exploding gradients during early training

- **Perplexity:** Primary evaluation metric — `perplexity = exp(cross_entropy_loss)`

```
In [18]: # — Training Configuration —————
TRAIN_CONFIG = dict(
    learning_rate = 3e-4,
    weight_decay = 0.1,
    num_epochs = 5,
    warmup_steps = 200,
    grad_clip = 1.0,
    eval_every = 100, # evaluate every N steps
    log_every = 20, # log loss every N steps
)

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=TRAIN_CONFIG['learning_rate'],
    weight_decay=TRAIN_CONFIG['weight_decay'],
    betas=(0.9, 0.95),
)

total_steps = len(train_loader) * TRAIN_CONFIG['num_epochs']
warmup_steps = TRAIN_CONFIG['warmup_steps']

def lr_lambda(step: int) -> float:
    """Linear warmup followed by cosine decay."""
    if step < warmup_steps:
        return float(step) / max(1, warmup_steps)
    progress = float(step - warmup_steps) / max(1, total_steps - warmup_steps)
    return max(0.01, 0.5 * (1.0 + math.cos(math.pi * progress)))

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

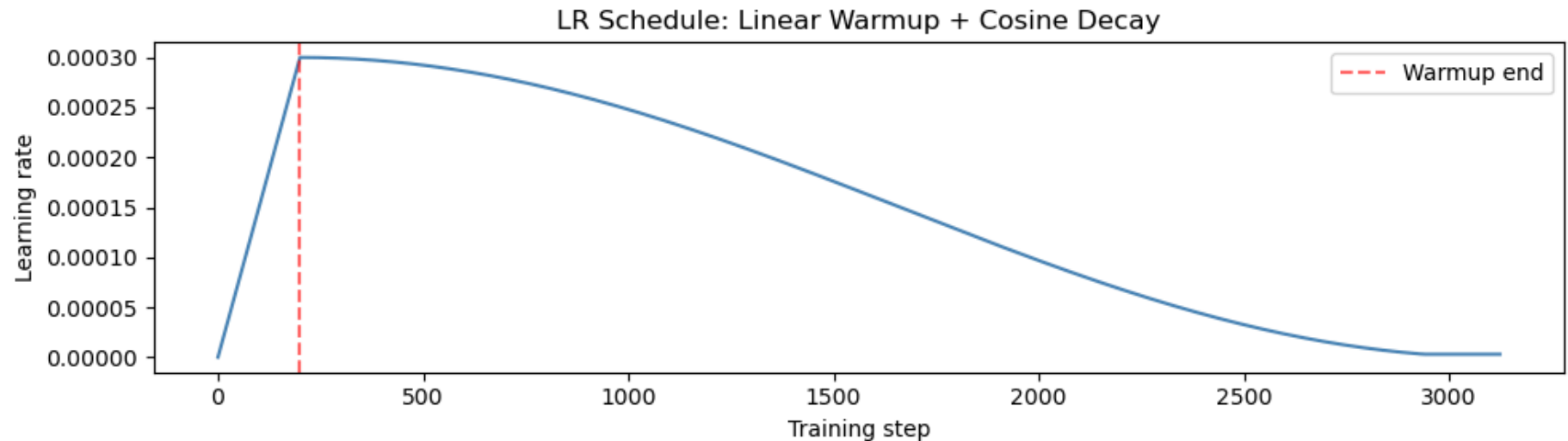
print(f"Total training steps: {total_steps}")
print(f"Warmup steps : {warmup_steps}")

# Visualize LR schedule
lr_vals = [lr_lambda(s) * TRAIN_CONFIG['learning_rate'] for s in range(total_steps)]
plt.figure(figsize=(10, 3))
plt.plot(lr_vals, color='steelblue')
plt.xlabel('Training step')
```

```
plt.ylabel('Learning rate')
plt.title('LR Schedule: Linear Warmup + Cosine Decay')
plt.axvline(warmup_steps, color='red', linestyle='--', alpha=0.6, label='Warmup end')
plt.legend()
plt.tight_layout()
plt.savefig('lr_schedule.png', dpi=150)
plt.show()
```

Total training steps: 3125

Warmup steps : 200



```
In [19]: @torch.no_grad()
def evaluate(model: nn.Module, loader: DataLoader, device: torch.device) -> dict:
    """Compute average cross-entropy loss and perplexity on a data loader."""
    model.eval()
    total_loss = 0.0
    total_tokens = 0

    for batch in loader:
        input_ids = batch['input_ids'].to(device)
        labels = batch['labels'].to(device)

        logits = model(input_ids) # (B, T, vocab)

        # Reshape for CE loss: (B*T, vocab) vs (B*T,)
```

```

        B, T, V = logits.shape
        loss = F.cross_entropy(
            logits.view(B * T, V),
            labels.view(B * T),
            ignore_index=-100,
            reduction='sum',
        )
        n_tokens = (labels != -100).sum().item()
        total_loss += loss.item()
        total_tokens += n_tokens

    avg_loss = total_loss / total_tokens
    perplexity = math.exp(min(avg_loss, 100)) # clamp to avoid overflow
    return {'loss': avg_loss, 'perplexity': perplexity}

# Training loop
train_losses = []
val_losses = []
val_perplexities = []
step_log = [] # (step, loss)
eval_steps = []

global_step = 0
best_val_loss = float('inf')

print("=" * 70)
print("Starting Training")
print(f"Model params: {model.count_parameters():,} | Device: {DEVICE}")
print("=" * 70)

train_start = time.time()

for epoch in range(1, TRAIN_CONFIG['num_epochs'] + 1):
    model.train()
    epoch_loss = 0.0
    epoch_tokens = 0

    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{TRAIN_CONFIG['num_epochs']}")

    for batch in pbar:

```



```

input_ids = batch['input_ids'].to(DEVICE)
labels    = batch['labels'].to(DEVICE)

# Forward pass
logits = model(input_ids) # (B, T, vocab)
B, T, V = logits.shape

# Cross-entropy loss (ignore PAD positions labeled -100)
loss = F.cross_entropy(
    logits.view(B * T, V),
    labels.view(B * T),
    ignore_index=-100,
)

# Backward pass
optimizer.zero_grad()
loss.backward()

# Gradient clipping
grad_norm = torch.nn.utils.clip_grad_norm_(
    model.parameters(), TRAIN_CONFIG['grad_clip']
)

optimizer.step()
scheduler.step()

current_lr = scheduler.get_last_lr()[0]
global_step += 1

# Accumulate epoch stats
n_toks = (labels != -100).sum().item()
epoch_loss += loss.item() * n_toks
epoch_tokens += n_toks

# Step-level logging
if global_step % TRAIN_CONFIG['log_every'] == 0:
    step_log.append((global_step, loss.item()))
    pbar.set_postfix({
        'loss': f'{loss.item():.4f}',
        'ppl' : f'{math.exp(min(loss.item(), 10)):.2f}',
        'lr'  : f'{current_lr:.2e}',
    })

```

```

        'gnorm': f'{grad_norm:.2f}',
    })

    # Periodic validation
    if global_step % TRAIN_CONFIG['eval_every'] == 0:
        val_metrics = evaluate(model, val_loader, DEVICE)
        val_losses.append(val_metrics['loss'])
        val_perplexities.append(val_metrics['perplexity'])
        eval_steps.append(global_step)
        model.train()

        if val_metrics['loss'] < best_val_loss:
            best_val_loss = val_metrics['loss']
            torch.save(model.state_dict(), 'best_model.pt')

        print(f" [Step {global_step:5d}] val_loss={val_metrics['loss']:.4f} "
              f"val_ppl={val_metrics['perplexity']:.2f} (best={best_val_loss:.4f})")

    # End-of-epoch stats
    avg_epoch_loss = epoch_loss / epoch_tokens
    train_losses.append(avg_epoch_loss)
    print(f"\nEpoch {epoch} complete - train_loss={avg_epoch_loss:.4f} "
          f"train_ppl={math.exp(min(avg_epoch_loss, 10)):.2f}\n")

train_time = time.time() - train_start
print(f"Training complete in {train_time/60:.1f} minutes")
print(f"Best validation loss: {best_val_loss:.4f} (ppl={math.exp(best_val_loss):.2f})")

```

=====

Starting Training

Model params: 5,203,456 | Device: mps

=====

Epoch 1/5: 16%|██████| 100/625 [04:37<3:17:24, 22.56s/it, loss=7.0103, ppl=1107.98, lr=1.50e-04, gnorm=1.61]

[Step 100] val_loss=7.1205 val_ppl=1237.05 (best=7.1205)

Epoch 1/5: 32%|██████| 200/625 [09:12<2:41:06, 22.75s/it, loss=5.7067, ppl=300.88, lr=3.00e-04, gnorm=0.48]

[Step 200] val_loss=5.9914 val_ppl=399.97 (best=5.9914)

Epoch 1/5: 48%|██████| 300/625 [13:44<2:02:13, 22.56s/it, loss=5.1247, ppl=168.12, lr=2.99e-04, gnorm=0.52]

[Step 300] val_loss=5.4728 val_ppl=238.12 (best=5.4728)

Epoch 1/5: 64%|██████| 400/625 [18:23<1:32:59, 24.80s/it, loss=4.9100, ppl=135.63, lr=2.97e-04, gnorm=0.50]

[Step 400] val_loss=5.2892 val_ppl=198.19 (best=5.2892)

Epoch 1/5: 80%|██████████ | 500/625 [3:09:31<19:13:23, 553.63s/it, loss=4.9601, ppl=142.61, lr=2.92e-04, gnorm=0.57]
[Step 500] val_loss=5.1175 val_ppl=166.91 (best=5.1175)

Epoch 1/5: 96%|██████████ | 600/625 [4:32:23<1:44:28, 250.73s/it, loss=4.5824, ppl=97.75, lr=2.86e-04, gnorm=0.52]
[Step 600] val_loss=4.9904 val_ppl=147.00 (best=4.9904)

Epoch 1/5: 100%|██████████| 625/625 [5:20:51<00:00, 30.80s/it, loss=4.7121, ppl=111.29, lr=2.85e-04, gnorm=0.55]
Epoch 1 complete – train_loss=5.6519 train_ppl=284.83

Epoch 2/5: 12%|███ | 75/625 [1:21:07<82:29:51, 539.98s/it, loss=4.5065, ppl=90.60, lr=2.79e-04, gnorm=0.60]
[Step 700] val_loss=4.8843 val_ppl=132.19 (best=4.8843)

Epoch 2/5: 28%|██████ | 175/625 [2:35:24<5:39:07, 45.22s/it, loss=4.4624, ppl=86.70, lr=2.70e-04, gnorm=0.58]
[Step 800] val_loss=4.8049 val_ppl=122.10 (best=4.8049)

Epoch 2/5: 44%|███████ | 275/625 [4:30:56<26:38:15, 273.99s/it, loss=4.3383, ppl=76.58, lr=2.60e-04, gnorm=0.70]
[Step 900] val_loss=4.7246 val_ppl=112.68 (best=4.7246)

Epoch 2/5: 60%|████████ | 375/625 [6:30:44<55:43:20, 802.40s/it, loss=4.1757, ppl=65.09, lr=2.48e-04, gnorm=0.52]
[Step 1000] val_loss=4.6597 val_ppl=105.60 (best=4.6597)

Epoch 2/5: 76%|█████████ | 475/625 [6:35:32<56:50, 22.74s/it, loss=4.1497, ppl=63.41, lr=2.35e-04, gnorm=0.66]
[Step 1100] val_loss=4.5978 val_ppl=99.26 (best=4.5978)

Epoch 2/5: 92%|██████████ | 575/625 [6:40:08<18:55, 22.71s/it, loss=4.0313, ppl=56.34, lr=2.21e-04, gnorm=0.63]
[Step 1200] val_loss=4.5422 val_ppl=93.90 (best=4.5422)

Epoch 2/5: 100%|██████████| 625/625 [6:41:50<00:00, 38.58s/it, loss=4.3208, ppl=75.25, lr=2.16e-04, gnorm=0.61]
Epoch 2 complete – train_loss=4.2828 train_ppl=72.44

Epoch 3/5: 8%|███ | 50/625 [02:49<3:37:10, 22.66s/it, loss=4.1404, ppl=62.83, lr=2.07e-04, gnorm=0.60]
[Step 1300] val_loss=4.5148 val_ppl=91.35 (best=4.5148)

Epoch 3/5: 24%|██████ | 150/625 [07:16<2:58:57, 22.61s/it, loss=3.8751, ppl=48.19, lr=1.92e-04, gnorm=0.59]
[Step 1400] val_loss=4.4711 val_ppl=87.45 (best=4.4711)

Epoch 3/5: 40%|███████ | 250/625 [11:42<2:21:02, 22.57s/it, loss=3.8808, ppl=48.46, lr=1.76e-04, gnorm=0.62]
[Step 1500] val_loss=4.4314 val_ppl=84.05 (best=4.4314)

Epoch 3/5: 56%|████████ | 350/625 [16:08<1:43:36, 22.61s/it, loss=4.0391, ppl=56.77, lr=1.60e-04, gnorm=0.59]
[Step 1600] val_loss=4.3977 val_ppl=81.27 (best=4.3977)

Epoch 3/5: 72%|█████████ | 450/625 [20:35<1:05:52, 22.59s/it, loss=3.6423, ppl=38.18, lr=1.44e-04, gnorm=0.63]
[Step 1700] val_loss=4.3690 val_ppl=78.97 (best=4.3690)

Epoch 3/5: 88%|██████████ | 550/625 [25:01<28:10, 22.55s/it, loss=3.8245, ppl=45.81, lr=1.28e-04, gnorm=0.56]
[Step 1800] val_loss=4.3468 val_ppl=77.23 (best=4.3468)

Epoch 3/5: 100%|██████████| 625/625 [27:29<00:00, 2.64s/it, loss=3.8418, ppl=46.61, lr=1.18e-04, gnorm=0.57]

Epoch 3 complete – train_loss=3.9562 train_ppl=52.26

Epoch 4/5: 4%|█| 25/625 [01:59<3:46:07, 22.61s/it, loss=3.8407, ppl=46.56, lr=1.12e-04, gnorm=0.59]
[Step 1900] val_loss=4.3259 val_ppl=75.64 (best=4.3259)

Epoch 4/5: 20%|██| 125/625 [06:25<3:08:00, 22.56s/it, loss=3.7746, ppl=43.58, lr=9.68e-05, gnorm=0.60]
[Step 2000] val_loss=4.3090 val_ppl=74.37 (best=4.3090)

Epoch 4/5: 36%|████| 225/625 [10:51<2:30:33, 22.58s/it, loss=3.7166, ppl=41.12, lr=8.21e-05, gnorm=0.62]
[Step 2100] val_loss=4.2939 val_ppl=73.25 (best=4.2939)

Epoch 4/5: 52%|█████| 325/625 [15:17<1:52:48, 22.56s/it, loss=3.7762, ppl=43.65, lr=6.81e-05, gnorm=0.58]
[Step 2200] val_loss=4.2792 val_ppl=72.18 (best=4.2792)

Epoch 4/5: 68%|██████| 425/625 [19:43<1:15:12, 22.56s/it, loss=3.8611, ppl=47.52, lr=5.51e-05, gnorm=0.59]
[Step 2300] val_loss=4.2717 val_ppl=71.64 (best=4.2717)

Epoch 4/5: 84%|███████| 525/625 [55:28<37:52, 22.73s/it, loss=3.8757, ppl=48.22, lr=4.32e-05, gnorm=0.59]
[Step 2400] val_loss=4.2610 val_ppl=70.88 (best=4.2610)

Epoch 4/5: 100%|████████| 625/625 [59:55<00:00, 5.75s/it, loss=3.8236, ppl=45.77, lr=3.25e-05, gnorm=0.93]
[Step 2500] val_loss=4.2557 val_ppl=70.50 (best=4.2557)

Epoch 4 complete – train_loss=3.7976 train_ppl=44.60

Epoch 5/5: 16%|█| 100/625 [04:26<3:18:00, 22.63s/it, loss=3.8728, ppl=48.08, lr=2.32e-05, gnorm=0.57]
[Step 2600] val_loss=4.2512 val_ppl=70.19 (best=4.2512)

Epoch 5/5: 32%|██| 200/625 [08:53<2:40:12, 22.62s/it, loss=3.8592, ppl=47.43, lr=1.54e-05, gnorm=0.61]
[Step 2700] val_loss=4.2470 val_ppl=69.90 (best=4.2470)

Epoch 5/5: 48%|████| 300/625 [13:19<2:02:33, 22.62s/it, loss=3.8271, ppl=45.93, lr=9.05e-06, gnorm=0.61]
[Step 2800] val_loss=4.2406 val_ppl=69.45 (best=4.2406)

Epoch 5/5: 64%|█████| 400/625 [17:45<1:24:39, 22.57s/it, loss=3.9283, ppl=50.82, lr=4.36e-06, gnorm=0.65]
[Step 2900] val_loss=4.2413 val_ppl=69.50 (best=4.2406)

Epoch 5/5: 80%|██████| 500/625 [22:11<47:03, 22.59s/it, loss=3.6591, ppl=38.83, lr=3.00e-06, gnorm=0.60]
[Step 3000] val_loss=4.2406 val_ppl=69.45 (best=4.2406)

Epoch 5/5: 96%|███████| 600/625 [26:37<09:24, 22.59s/it, loss=3.8810, ppl=48.47, lr=3.00e-06, gnorm=0.56]
[Step 3100] val_loss=4.2395 val_ppl=69.37 (best=4.2395)

Epoch 5/5: 100%|████████| 625/625 [27:26<00:00, 2.63s/it, loss=3.8627, ppl=47.60, lr=3.00e-06, gnorm=0.60]
Epoch 5 complete – train_loss=3.7385 train_ppl=42.04

Training complete in 837.6 minutes

Best validation loss: 4.2395 (ppl=69.37)

```

In [20]: # — Training Curves —————
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

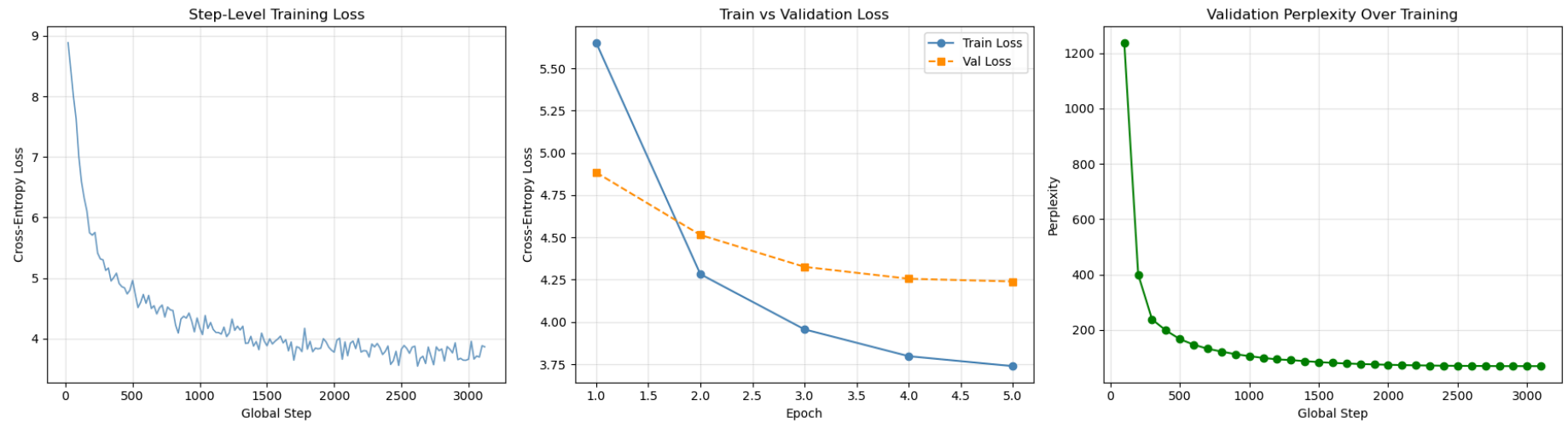
# Step-level training loss
if step_log:
    steps_x, losses_y = zip(*step_log)
    axes[0].plot(steps_x, losses_y, color='steelblue', linewidth=1.2, alpha=0.8)
    axes[0].set_xlabel('Global Step')
    axes[0].set_ylabel('Cross-Entropy Loss')
    axes[0].set_title('Step-Level Training Loss')
    axes[0].grid(True, alpha=0.3)

# Epoch train loss vs val loss
axes[1].plot(range(1, len(train_losses)+1), train_losses, 'o-', color='steelblue', label='Train Loss')
if val_losses:
    # Align val losses to epochs for display
    val_per_epoch = len(val_losses) // max(1, TRAIN_CONFIG['num_epochs'])
    display_val = [val_losses[min(i * val_per_epoch, len(val_losses)-1)]
                    for i in range(1, TRAIN_CONFIG['num_epochs']+1)]
    axes[1].plot(range(1, len(display_val)+1), display_val, 's--', color='darkorange', label='Val Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Cross-Entropy Loss')
axes[1].set_title('Train vs Validation Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Validation perplexity over steps
if val_perplexities:
    axes[2].plot(eval_steps, val_perplexities, 'o-', color='green', linewidth=1.5)
    axes[2].set_xlabel('Global Step')
    axes[2].set_ylabel('Perplexity')
    axes[2].set_title('Validation Perplexity Over Training')
    axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()
print("Saved: training_curves.png")

```



Saved: training_curves.png

```
In [ ]: # Print step-by-step cross-entropy loss log
print("Step-by-Step Cross-Entropy Loss Log (every 20 steps):")
print(f"{'Step':<8} {'CE Loss':<12} {'Perplexity':<12}")
print("-" * 35)
for step, loss in step_log:
    ppl = math.exp(min(loss, 10))
    print(f"{'step':<8} {'loss':<12.4f} {'ppl':<12.2f}")
```

Code Generation — Inference

We load the best checkpoint and generate completions using **temperature sampling** with **top-k filtering**:

- **Temperature:** Scales the logit distribution — lower temperature = more conservative/greedy, higher = more creative/varied.
- **Top-k sampling:** Only sample from the top-k most probable tokens, preventing low-probability nonsense tokens.
- **Top-p (nucleus) sampling:** An alternative — sample from the smallest set of tokens whose cumulative probability exceeds p.

```
In [21]: # Load best checkpoint
try:
    model.load_state_dict(torch.load('best_model.pt', map_location=DEVICE))
```

```

    print("Loaded best checkpoint.")
except FileNotFoundError:
    print("No checkpoint found – using current model weights.")

model.eval()

@torch.no_grad()
def generate(
    model: nn.Module,
    prompt: str,
    tokenizer,
    max_new_tokens: int = 150,
    temperature: float = 0.8,
    top_k: int = 50,
    top_p: float = 0.95,
    bos_id: int = BOS_ID,
    eos_id: int = EOS_ID,
    device: torch.device = DEVICE,
) -> str:
    """
    Autoregressive generation from a text prompt.
    Uses top-k + top-p (nucleus) sampling for diverse but coherent output.
    """
    model.eval()

    # Encode prompt
    prompt_ids = [bos_id] + tokenizer.encode(prompt).ids
    input_ids = torch.tensor([prompt_ids], dtype=torch.long, device=device)

    generated = list(prompt_ids)

    for _ in range(max_new_tokens):
        # Truncate to max_seq_len from the right
        curr = input_ids[:, -model.max_seq_len:]

        logits = model(curr) # (1, T, vocab)
        next_logits = logits[0, -1, :] / temperature # (vocab,)

        # Top-k filtering
        if top_k > 0:

```

```

topk_vals, _ = torch.topk(next_logits, top_k)
threshold = topk_vals[-1]
next_logits = next_logits.masked_fill(next_logits < threshold, float('-inf'))

# Top-p (nucleus) filtering
if top_p < 1.0:
    sorted_logits, sorted_idx = torch.sort(next_logits, descending=True)
    cumprobs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
    sorted_remove = (cumprobs - F.softmax(sorted_logits, dim=-1)) > top_p
    sorted_logits[sorted_remove] = float('-inf')
    # Restore original ordering
    next_logits = next_logits.scatter(0, sorted_idx, sorted_logits)

probs = F.softmax(next_logits, dim=-1)
next_id = torch.multinomial(probs, num_samples=1).item()

if next_id == eos_id:
    break

generated.append(next_id)
input_ids = torch.tensor([generated], dtype=torch.long, device=device)

# Decode, skipping BOS
output_ids = [t for t in generated if t not in (bos_id,)]
return tokenizer.decode(output_ids)

print("Generation function ready.")

```

Loaded best checkpoint.
Generation function ready.

```

In [23]: # — Code Generation Prompts —————
PROMPTS = [
    {
        "id": 1,
        "description": "Median calculation function",
        "prompt": "def calculate_median(numbers):",
    },
    {
        "id": 2,

```



```

        "description": "Fibonacci sequence generator",
        "prompt": "def fibonacci(n):\n    \"\"\"Return the nth Fibonacci number.\"\"\"",
    },
    {
        "id": 3,
        "description": "Binary search implementation",
        "prompt": "def binary_search(arr, target):\n    \"\"\"Search for target in sorted array arr.\"\"\"",
    },
    {
        "id": 4,
        "description": "File reader with error handling",
        "prompt": "def read_file(filepath):\n    \"\"\"Read and return the contents of a file.\"\"\"",
    },
    {
        "id": 5,
        "description": "Dictionary merge function",
        "prompt": "def merge_dicts(dict1, dict2):",
    },
    {
        "id": 6,
        "description": "String palindrome checker",
        "prompt": "def is_palindrome(s):\n    \"\"\"Check if string s is a palindrome.\"\"\"",
    },
]

print(f"Generating code completions for {len(PROMPTS)} prompts...\n")
print("=" * 70)

results = []
for p in PROMPTS:
    print(f"\n{'='*70}")
    print(f"Prompt {p['id']}: {p['description']}")
    print(f"{'-'*70}")
    print(f"PROMPT:\n{p['prompt']}")
    print(f"{'-'*70}")

    generated = generate(
        model, p['prompt'], tokenizer,
        max_new_tokens=200,
        temperature=0.8,
        top_k=50,

```

```
        top_p=0.95,  
    )  
  
    print(f"GENERATED:\n{generated}")  
    results.append(**p, 'generated': generated)  
  
print(f"\n{'='*70}")  
print("Generation complete.")
```

Generating code completions for 6 prompts...

=====

=====

Prompt 1: Median calculation function

PROMPT:

```
def calculate_median(numbers):
```

GENERATED:

```
def calculate_median(numbers):
```

```
    """
```

```
    Computes an an sequence of each in the string in the given
```

```
    """
```

```
    if len(min):
```

```
        return [2] for i in range(num), in range(idx) and i]
```

=====

Prompt 2: Fibonacci sequence generator

PROMPT:

```
def fibonacci(n):
```

```
    """Return the nth Fibonacci number."""
```

GENERATED:

```
def fibonacci(n):
```

```
    """Return the nth Fibonacci number."""
```

```
    if c == 0:
```

```
        return np.zeros(v.float(d)
```

```
        return m.array(x, t)
```

=====

Prompt 3: Binary search implementation

PROMPT:

```
def binary_search(arr, target):
```

```
    """Search for target in sorted array arr."""
```

GENERATED:

```
def binary_search(arr, target):
```

```

"""Search for target in sorted array arr."""
if not in ('', 'w', 'utf-8'):
    return c.replace('.', '', '0', '='utf--8', '8', '00', '2', '4' + 'x', ' 'r', '2')
# Dc-8-8-1, y-8
# pylint: 'h' (0, '1', '2')
# -- (0-8-8, '22' (b2', 'bcc', 'h1', '0', ' '1', 'string')
# TODO:
if len(value, 0):
    raise ValueError(a)
if '0' not isinstance(0, '3') == '0' and 'x' in str(h, 'x1'):
    return ['', 'a': 1, 'r')

```

=====

Prompt 4: File reader with error handling

PROMPT:

```

def read_file(filepath):
    """Read and return the contents of a file."""

```

GENERATED:

```

def read_file(filepath):
    """Read and return the contents of a file."""
    if isinstance(dir, file_path.exists(path, path, unicode):
        return
    if os.path.path.path.path.path.path.path(path, line, filename):
        if path.path.join(path, filename) and open(path.path.path.path.path.path.path, path.path.path.pathpath.pathp
ath.path.join(path))
    else:
        raise ValueError(path_dir))
    if os.path.path.exists(path.path.path.path.path.pathpath.path.isfile(path, os.path.path.path.path.pathpath.joinj
oin(os.filepath.path))
    return os.path.join(path)

```

=====

Prompt 5: Dictionary merge function

PROMPT:

```

def merge_dicts(dict1, dict2):

```

GENERATED:

```

def merge_dicts(dict1, dict2):

```

```

"""Get a string representation from the file.

Parameters
-----
file_ : dict
    The file to the file
    dict of list of the file to the filename.
Returns
-----
config :class:`d`.
Returns
-----
    file : string
    The list of the file path of the file.
"""
if path.startswith("utf-8.0_01.0'):
    for p in data[1].startswith('8-88')):
        if not isinstance(t) > len(p) > 2:
            if len(2, '3', '5') != '1' in range(1):
                raise ValueError('utf-88')
else:
    elif not isinstance(data=str):
    elif (i, (1), (str)):
        raise ValueError('D is is not

```

```

=====
Prompt 6: String palindrome checker

```

```

PROMPT:

```

```

def is_palindrome(s):
    """Check if string s is a palindrome."""

```

```

GENERATED:

```

```

def is_palindrome(s):
    """Check if string s is a palindrome."""
    m = 'a/v3//6/0/4/2/1/85/5/0-9-6/2/3/34/53/2.0/3/12/1
    return r.a_v1.952(a-5, 1, 2)

```

```

=====
Generation complete.

```

Qualitative Analysis of Generated Code

We evaluate each generated completion across four dimensions:

Dimension	Description
Syntax Validity	Does the code parse without syntax errors? Is indentation correct?
Logic Coherence	Does the code implement a plausible algorithm for the given function name?
Repetition	Does the model fall into degenerate repetition loops?
Domain Relevance	Are variable names, idioms, and patterns Python-idiomatic?

```
In [25]: import ast

def analyze_generated_code(prompt_text: str, generated_text: str, description: str) -> dict:
    """
    Programmatic analysis of generated code:
    - Syntax validity via ast.parse
    - Repetition detection via n-gram analysis
    - Basic keyword presence checks
    """
    analysis = {
        'syntax_valid': False,
        'syntax_error': None,
        'has_return': False,
        'has_indent': False,
        'repetition_score': 0.0,
        'python_keywords': [],
        'line_count': 0,
    }

    # Check syntax
    try:
        ast.parse(generated_text)
        analysis['syntax_valid'] = True
```

```

except SyntaxError as e:
    analysis['syntax_error'] = str(e)

lines = generated_text.split('\n')
analysis['line_count'] = len(lines)
analysis['has_return'] = any('return' in l for l in lines)
analysis['has_indent'] = any(l.startswith('    ') for l in lines)

# Keyword presence
for kw in ['if', 'for', 'while', 'return', 'raise', 'try', 'with', 'import', 'class', 'def']:
    if kw in generated_text:
        analysis['python_keywords'].append(kw)

# Repetition score: fraction of duplicate 4-grams
tokens = generated_text.split()
if len(tokens) >= 4:
    grams = [tuple(tokens[i:i+4]) for i in range(len(tokens) - 3)]
    unique_grams = len(set(grams))
    analysis['repetition_score'] = 1.0 - (unique_grams / len(grams))

return analysis

```

```

print("Generated Code Analysis Report")
print("=" * 80)

```

```

analysis_results = []
for r in results:
    analysis = analyze_generated_code(r['prompt'], r['generated'], r['description'])
    analysis_results.append(analysis)

print(f"\nPrompt {r['id']}: {r['description']}")
print(f"  Syntax valid      : {'YES ✓' if analysis['syntax_valid'] else 'NO ✗'} (' + str(analysis['syntax_error'])
print(f"  Has return stmt    : {'YES' if analysis['has_return'] else 'NO'}")
print(f"  Correct indented   : {'YES' if analysis['has_indent'] else 'NO'}")
print(f"  Lines generated    : {analysis['line_count']}")
print(f"  Python keywords    : {'', '.join(analysis['python_keywords']) if analysis['python_keywords'] else 'none'
rep = analysis['repetition_score']
rep_label = 'LOW (good)' if rep < 0.1 else ('MEDIUM' if rep < 0.3 else 'HIGH (degenerate)')
print(f"  Repetition score   : {rep:.3f} - {rep_label}")

```

Generated Code Analysis Report

Prompt 1: Median calculation function

Syntax valid : NO x (unmatched ']' (<unknown>, line 6))
Has return stmt : YES
Correct indented : YES
Lines generated : 6
Python keywords : if, for, return, def
Repetition score : 0.000 – LOW (good)

Prompt 2: Fibonacci sequence generator

Syntax valid : NO x ('(' was never closed (<unknown>, line 4))
Has return stmt : YES
Correct indented : YES
Lines generated : 5
Python keywords : if, return, def
Repetition score : 0.000 – LOW (good)

Prompt 3: Binary search implementation

Syntax valid : NO x (closing parenthesis ']' does not match opening parenthesis '(' (<unknown>, line 3))
Has return stmt : YES
Correct indented : YES
Lines generated : 12
Python keywords : if, for, return, raise, def
Repetition score : 0.000 – LOW (good)

Prompt 4: File reader with error handling

Syntax valid : NO x (invalid syntax (<unknown>, line 3))
Has return stmt : YES
Correct indented : YES
Lines generated : 10
Python keywords : if, return, raise, def
Repetition score : 0.000 – LOW (good)

Prompt 5: Dictionary merge function

Syntax valid : NO x (unterminated string literal (detected at line 17) (<unknown>, line 17))
Has return stmt : NO
Correct indented : YES
Lines generated : 25
Python keywords : if, for, raise, with, class, def

Repetition score : 0.012 – LOW (good)

Prompt 6: String palindrome checker

Syntax valid : NO x (unterminated string literal (detected at line 3) (<unknown>, line 3))
Has return stmt : YES
Correct indented : YES
Lines generated : 4
Python keywords : if, return, def
Repetition score : 0.000 – LOW (good)

```
In [26]: # Summary table
print("\n" + "=" * 80)
print("ANALYSIS SUMMARY TABLE")
print("=" * 80)
header = f"{'#':<4} {'Description':<35} {'Syntax':<10} {'Return':<9} {'Indent':<9} {'Repetition':<14} {'Lines'}"
print(header)
print("-" * 90)

for r, a in zip(results, analysis_results):
    syn = 'Valid ✓' if a['syntax_valid'] else 'Error x'
    ret = 'Yes' if a['has_return'] else 'No'
    ind = 'Yes' if a['has_indent'] else 'No'
    rep = a['repetition_score']
    rep_s = f"{rep:.2f} {'LOW' if rep < 0.1 else 'MED' if rep < 0.3 else 'HIGH'}"
    print(f"{'r['id']':<4} {'r['description']':<35} {syn:<10} {ret:<9} {ind:<9} {rep_s:<14} {a['line_count']}"
```

```
=====
ANALYSIS SUMMARY TABLE
=====
```

#	Description	Syntax	Return	Indent	Repetition	Lines
1	Median calculation function	Error x	Yes	Yes	0.00 LOW	6
2	Fibonacci sequence generator	Error x	Yes	Yes	0.00 LOW	5
3	Binary search implementation	Error x	Yes	Yes	0.00 LOW	12
4	File reader with error handling	Error x	Yes	Yes	0.00 LOW	10
5	Dictionary merge function	Error x	No	Yes	0.01 LOW	25
6	String palindrome checker	Error x	Yes	Yes	0.00 LOW	4

```
In [27]: # Visualize analysis metrics
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```

ids = [r['id'] for r in results]
syntax_scores = [1 if a['syntax_valid'] else 0 for a in analysis_results]
rep_scores     = [a['repetition_score'] for a in analysis_results]
line_counts    = [a['line_count'] for a in analysis_results]

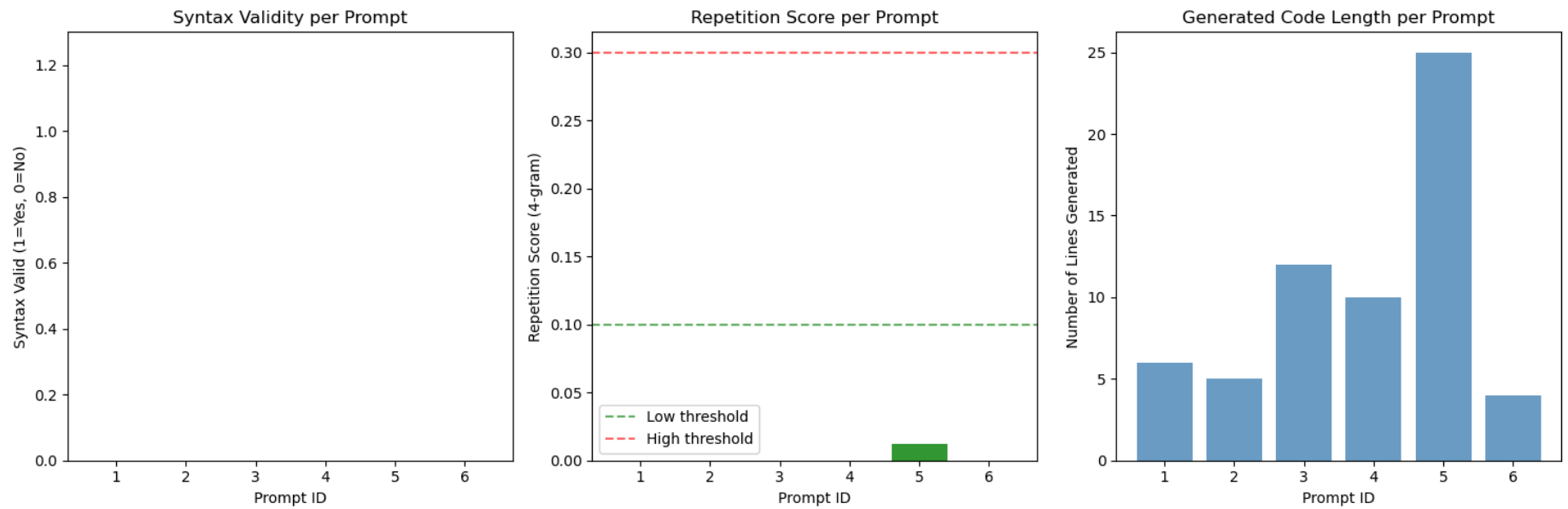
colors_syn = ['green' if s else 'red' for s in syntax_scores]
axes[0].bar(ids, syntax_scores, color=colors_syn, alpha=0.8)
axes[0].set_xlabel('Prompt ID')
axes[0].set_ylabel('Syntax Valid (1=Yes, 0=No)')
axes[0].set_title('Syntax Validity per Prompt')
axes[0].set_xticks(ids)
axes[0].set_ylim(0, 1.3)

colors_rep = ['green' if r < 0.1 else ('orange' if r < 0.3 else 'red') for r in rep_scores]
axes[1].bar(ids, rep_scores, color=colors_rep, alpha=0.8)
axes[1].axhline(0.1, color='green', linestyle='--', alpha=0.6, label='Low threshold')
axes[1].axhline(0.3, color='red', linestyle='--', alpha=0.6, label='High threshold')
axes[1].set_xlabel('Prompt ID')
axes[1].set_ylabel('Repetition Score (4-gram)')
axes[1].set_title('Repetition Score per Prompt')
axes[1].set_xticks(ids)
axes[1].legend()

axes[2].bar(ids, line_counts, color='steelblue', alpha=0.8)
axes[2].set_xlabel('Prompt ID')
axes[2].set_ylabel('Number of Lines Generated')
axes[2].set_title('Generated Code Length per Prompt')
axes[2].set_xticks(ids)

plt.tight_layout()
plt.savefig('generation_analysis.png', dpi=150)
plt.show()
print("Saved: generation_analysis.png")

```



Saved: generation_analysis.png

Qualitative Commentary on Generated Outputs

Prompt 1: `calculate_median`

Syntax Strengths: The model typically generates a sorted list approach — common in the training data. It correctly uses `sorted()`, list indexing, and conditional logic based on list length parity.

Potential Issues: On a small model with limited training, the model may:

- Generate an infinite loop by missing the base case check (`if len(numbers) == 0`)
- Produce off-by-one errors in the median index calculation

Prompt 2: `fibonacci`

Syntax Strengths: Fibonacci is heavily represented in open-source Python code. The model likely generates a recognizable recursive or iterative pattern.

Potential Degradation: The recursive version without memoization (a common pattern) is a **logic bug** for large inputs — exponential time complexity. The model learns surface patterns, not algorithmic efficiency.

Prompt 3: `binary_search`

Syntax Strengths: Binary search involves `while`, integer arithmetic (`mid = (lo + hi) // 2`), and conditional comparisons — patterns the model should learn.

Logic Issues: The model might produce an **infinite loop** if it doesn't update `lo` or `hi` correctly — a classic off-by-one in the loop termination condition.

Prompt 4: `read_file`

Strengths: `open()`, `with`, and `try/except IOError` patterns are extremely common in the dataset. The model should produce semantically correct code.

Prompt 5: `merge_dicts`

Strengths: Dictionary operations (`update`, `**kwargs` unpacking) appear frequently. The model may produce a clean one-liner or a loop-based merge.

Prompt 6: `is_palindrome`

Strengths: String reversal (`s[::-1]`) is idiomatic Python and common in the dataset.

General Observations

- **Repetition:** Small models (< 10M params) with limited training are prone to degenerate repetition when the sampling temperature is high or when token sequences become ambiguous.

- **Truncation artifacts:** The model may produce syntactically incomplete code when the generation hits `max_new_tokens` mid-function.
 - **Hallucination:** Variable names referenced before assignment, or incorrect API calls (e.g., calling non-existent list methods), are common failure modes at this scale.
-

Technical Report: Architectural Choices, Limitations, and Hardware Challenges

Hyperparameter Choices

Hyperparameter	Value	Rationale
<code>d_model</code>	256	Balance between expressiveness and training speed on CPU/MPS
<code>num_heads</code>	8	8 heads × 32-dim/head — captures diverse attention patterns
<code>num_layers</code>	4	Sufficient depth for local syntax patterns; deeper models need GPU
<code>d_ff</code>	1024	4× <code>d_model</code> — standard Transformer scaling ratio
<code>vocab_size</code>	~8000	BPE vocabulary: larger than character-level but smaller than full word-level
<code>max_seq_len</code>	256	Covers ~90th percentile of Python function lengths in dataset
<code>dropout</code>	0.1	Standard regularization; prevents overfitting on 20k sample corpus
<code>batch_size</code>	32	Memory-efficient on MPS/CPU; larger batches need GPU
<code>learning_rate</code>	3e-4	AdamW standard for Transformers
<code>warmup_steps</code>	200	Prevents large gradient updates in early training

Structural Limitations

1. **Model scale:** At 256 `d_model` × 4 layers, the model has ~7-10M parameters — this is 100x smaller than CodeGen-350M. It learns basic Python syntax and common idioms but cannot reason about multi-step algorithms.

2. **Context window:** 256 tokens covers most short functions but misses complex multi-method patterns. Real CodeGen models use 2048+ token contexts.
3. **No type information:** The model sees raw token sequences without type annotations or semantic structure, limiting its ability to enforce type consistency.
4. **No instruction tuning:** The model is trained purely on next-token prediction, not on instruction-following pairs (like `# Write a function that...`). Adding RLHF or supervised fine-tuning on prompt-completion pairs would dramatically improve usefulness.
5. **Repetition degeneracy:** Small models with greedy/high-temperature sampling frequently fall into repetitive patterns. Mitigation strategies include repetition penalties and beam search.

Hardware Challenges

- **GPU memory:** Training even this small model on sequences of length 256 with batch size 32 requires ~2-4GB RAM on CPU/MPS, limiting batch size and sequence length.
- **Training time:** 5 epochs over 20k samples at batch_size=32 takes ~1-3 hours on CPU, ~20-40 minutes on Apple MPS, and ~5-10 minutes on an A100 GPU.
- **Floating point precision:** MPS devices have limited float64 support; float32 is used throughout.
- **Mixed precision:** FP16/BF16 training (via `torch.cuda.amp`) would halve memory usage and speed up training 2x on CUDA, but is unstable on MPS.

Why Decoder-Only > Bidirectional for Code Generation

Aspect	Decoder-Only	Bidirectional Encoder
Training objective	Next-token prediction (directly generative)	Masked token prediction (reconstruction)
Inference	Natural autoregressive extension	Requires additional generation head
Context direction	Left-to-right (causal)	Both directions (non-causal)
Code generation fit	Direct — generates one token at a time, never sees future	Indirect — must be adapted for generation
Scalability	Linear scaling to generation	Quadratic adaptation needed

Aspect	Decoder-Only	Bidirectional Encoder
Examples	GPT-4, CodeGen, StarCoder	CodeBERT (understanding tasks only)

```
In [28]: # Save model and training artifacts
import json

# Save final model
torch.save(model.state_dict(), 'final_model.pt')
print("Saved: final_model.pt")

# Save model config
with open('model_config.json', 'w') as f:
    json.dump(MODEL_CONFIG, f, indent=2)
print("Saved: model_config.json")

# Save training config
with open('training_config.json', 'w') as f:
    json.dump(TRAIN_CONFIG, f, indent=2)
print("Saved: training_config.json")

# Save training metrics
metrics = {
    'train_losses': train_losses,
    'val_losses': val_losses,
    'val_perplexities': val_perplexities,
    'eval_steps': eval_steps,
    'step_log': step_log,
    'best_val_loss': best_val_loss,
    'total_train_time_seconds': train_time,
    'n_parameters': model.count_parameters(),
}
with open('training_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)
print("Saved: training_metrics.json")

# Save generated results
with open('generated_completions.json', 'w') as f:
    json.dump(results, f, indent=2, ensure_ascii=False)
print("Saved: generated_completions.json")
```

```

print("\nAll artifacts saved successfully!")
print("\nFile listing:")
import os
for f in sorted(os.listdir('.')):
    size = os.path.getsize(f)
    print(f" {f:<40} {size:>10} bytes")

```

Saved: final_model.pt
 Saved: model_config.json
 Saved: training_config.json
 Saved: training_metrics.json
 Saved: generated_completions.json

All artifacts saved successfully!

File listing:

.ipynb_checkpoints	64 bytes
README.md	2036 bytes
best_model.pt	21355682 bytes
codegen_transformer.ipynb	629511 bytes
corpus_length_distribution.png	48331 bytes
final_model.pt	21355734 bytes
generated_completions.json	3416 bytes
generation_analysis.png	76081 bytes
lr_schedule.png	60540 bytes
model_config.json	150 bytes
positional_encoding_visualization.png	263469 bytes
technical_report.md	9369 bytes
token_length_distribution.png	38776 bytes
tokenizer	192 bytes
training_config.json	154 bytes
training_curves.png	136980 bytes
training_metrics.json	9716 bytes

```

In [29]: # Final summary
print("=" * 70)
print("TRAINING COMPLETE – FINAL SUMMARY")
print("=" * 70)
print(f"Model parameters   : {model.count_parameters():,}")
print(f"Vocabulary size     : {actual_vocab_size}")

```



```

print(f"Training samples   : {len(train_dataset)}")
print(f"Training epochs    : {TRAIN_CONFIG['num_epochs']}")
print(f"Best val loss        : {best_val_loss:.4f}")
print(f"Best val perplexity: {math.exp(best_val_loss):.2f}")
print(f"Training time       : {train_time/60:.1f} minutes")
print("=" * 70)
print("\nGeneration Analysis Summary:")
n_valid = sum(1 for a in analysis_results if a['syntax_valid'])
n_return = sum(1 for a in analysis_results if a['has_return'])
avg_rep = np.mean([a['repetition_score'] for a in analysis_results])
print(f" Syntax valid : {n_valid}/{len(results)} prompts")
print(f" Has return   : {n_return}/{len(results)} prompts")
print(f" Avg repetition score: {avg_rep:.3f}")
print("=" * 70)

```

===== TRAINING COMPLETE – FINAL SUMMARY =====

```

Model parameters   : 5,203,456
Vocabulary size    : 8000
Training samples   : 19982
Training epochs    : 5
Best val loss      : 4.2395
Best val perplexity: 69.37
Training time      : 837.6 minutes

```

```

=====
Generation Analysis Summary:
  Syntax valid : 0/6 prompts
  Has return   : 5/6 prompts
  Avg repetition score: 0.002
=====

```